Rank 13 — Organizational Knowledge Graph

Capability: Organizational Knowledge Graph
Purpose: Enterprise-Wide Knowledge Map
KMS Area: Tier 2 — Organizational Intelligence

Overview

The Organizational Knowledge Graph should provide a connected view of the knowledge that already exists throughout the KMS.

Today, organizational knowledge naturally exists across several different objects:

documents, projects, topics, departments, people, entities, decisions, lessons and relationships.

Each object is useful individually. However, much of the organizational intelligence comes from understanding **how these objects connect**.

The PRD defines Rank 13 as an **"Enterprise-wide knowledge map."**

The objective is therefore not simply to create an attractive network visualization. The graph should become a reusable knowledge layer that allows both users and future AI capabilities to navigate relationships across the organization.

What the Current KMS Already Gives Us

The existing KMS is already relational enough to form the first version of a knowledge graph.

Potential graph nodes already exist conceptually as:

- Organization
- Department
- Project
- Document
- Topic
- Entity
- Person/Expert
- Decision
- Lesson

The database also already captures several relationships among these objects.

Examples include:

Document → Topic
Document → Project
Document → Entity
Document → Decision
Topic → Expert
Project → Lesson
Document → Document

Rank 12 will further enrich this by automatically discovering relationships that were not explicitly stored before.

Therefore, Rank 13 does not need to create knowledge from scratch.

Its primary responsibility should be to **normalize, expose, traverse and visualize the relationships already available throughout the KMS.**

1. Graph Representation

At the graph level, almost everything can be represented using two concepts:

Nodes

A node represents a knowledge object.

For example:

Document: Security Architecture
Project: Identity Modernization
Topic: Authentication
Expert: Jane Doe
Lesson: Token Expiration Failure

Edges

An edge represents a relationship between two nodes.

For example:

Security Architecture → HAS_TOPIC → Authentication
Security Architecture → BELONGS_TO → Identity Modernization
Authentication → HAS_EXPERT → Jane Doe
Identity Modernization → HAS_LESSON → Token Expiration Failure

This gives the application a consistent graph representation even though the underlying information originates from different Prisma models.

2. Proposed Architecture

The graph should initially be implemented as an **application-layer graph over the existing PostgreSQL/Prisma database**.

High-level flow:

Existing Prisma Models
↓
Knowledge Graph Service
↓
Normalize Nodes + Edges
↓
Apply RBAC
↓
Graph Traversal
↓
Graph API
↓
Knowledge Graph UI / AI Retrieval

This approach avoids introducing a large infrastructure change simply to deliver the first version.

3. Why I Would Not Introduce Neo4j Immediately

A dedicated graph database may eventually be valuable, but I would not make it a requirement for Rank 13.

The current KMS already stores its organizational knowledge and relationships in PostgreSQL.

Moving the same data into another database immediately would introduce:

synchronization logic, duplicate storage, additional deployment infrastructure, consistency issues and another permission boundary.

For the first implementation, the KMS can build graph responses from its existing relational models.

A dedicated graph database should be evaluated later if real usage demonstrates that the system requires deep, high-volume, variable-length graph traversal that PostgreSQL/application-level traversal cannot efficiently support.

This keeps the implementation proportional to the current product.

4. Knowledge Graph Service

The graph logic should be separated from API/UI code.

Conceptually, a service such as:

src/lib/knowledgeGraph.js

could be responsible for converting existing KMS records into normalized nodes and edges.

For example, the service might receive:

Organization ID
Starting node
Requested depth
Filters
Current user

and return:

authorized nodes + authorized edges

The API and UI then do not need to understand every underlying Prisma relationship individually.

5. Traversal Is More Important Than Visualization

A common implementation mistake would be to treat Rank 13 as primarily a graph visualization feature.

Visualization is useful, but the more important capability is **graph traversal**.

For example, suppose a user selects:

Authentication

The system should be capable of traversing outward:

Authentication
→ related documents
→ projects containing those documents
→ experts associated with the topic
→ decisions contained in those documents
→ lessons associated with those projects.

That connected navigation is what makes the graph useful as an enterprise knowledge map.

6. Bounded Subgraphs Instead of Loading Everything

The application should not attempt to send the organization's entire graph to the browser.

For a large enterprise, that would quickly become unusable.

Instead, the graph should operate around a starting point.

For example:

User searches “Authentication”
↓
Authentication Topic selected
↓
Load direct neighbors
↓
User expands Project X
↓
Load Project X neighbors

This **progressive expansion** keeps both the backend and UI manageable.

The first version could limit traversal to one or two hops and impose node/edge limits.

7. Integration with Sandeep's Rank 12

The relationship between Rank 12 and Rank 13 should be very clear.

Rank 12 — Relationship Discovery

Determines:

What hidden relationships exist?

It discovers and scores connections and records evidence.

Rank 13 — Knowledge Graph

Determines:

How do we represent and navigate all of those relationships across the organization?

Rank 13 should therefore consume Rank 12 output instead of recreating its discovery logic.

Conceptually:

Documents + Topics + Projects + Experts + Decisions + Lessons
+ Rank 12 discovered relationships
↓
Organizational Knowledge Graph

This also gives the two implementations a clean technical boundary.

8. Knowledge Graph API

A graph endpoint could conceptually look like:

GET /api/org/[orgId]/knowledge-graph

with parameters for starting node, type, project, topic and traversal depth.

The response should remain frontend-neutral:

```
{
  "nodes": [
    {
      "id": "topic-12",
      "type": "topic",
      "label": "Authentication"
```

```
    }
  ],
  "edges": [
    {
      "source": "doc-18",
      "target": "topic-12",
      "type": "HAS_TOPIC"
    }
  ]
}
```


This allows the same graph service to support a visual interface today and potentially AI retrieval later.

9. Knowledge Graph Interface

The UI should allow users to begin with something familiar rather than presenting an enormous network immediately.

A user might search:

“Authentication”

The interface could then show:

Authentication

↳ 12 Documents
↳ 3 Projects
↳ 4 Experts
↳ 2 Decisions
↳ 3 Lessons

Selecting a connection expands that portion of the graph.

Useful controls would include:

Search
Node-type filters
Department filter

Project filter
Relationship filter
Expand/collapse
Open source

Clicking a document should still take the user to the normal document experience rather than duplicating document functionality inside the graph.

10. Evidence and Trust

The graph should distinguish between relationships coming directly from structured data and relationships inferred by Rank 12.

For example:

Explicit relationship

Document → belongs to → Project X

Inferred relationship

Document A → related to → Document B
Confidence: 91%
Evidence: shared project + topic + entities

That distinction is important for enterprise trust.

Users should be able to understand where a connection came from rather than assuming every graph edge is an established organizational fact.

11. Security

RBAC is especially important because a graph can accidentally expose information indirectly.

For example, even hiding a confidential document's content while displaying:

Confidential Acquisition Plan → Project Phoenix

could disclose sensitive information.

Therefore, authorization needs to happen **before graph nodes and edges are returned**, not merely when the user tries to open the underlying document.

The graph should reuse the current organization, department, project and document authorization policies.

12. Future Integration with Conversational Retrieval

The longer-term value of Rank 13 goes beyond visualization.

The graph can become another context source for the conversational assistant.

Current retrieval is roughly:

Question → Hybrid/Semantic Retrieval → Relevant Chunks → LLM

Future retrieval could become:

Question
↓
Hybrid/Semantic Retrieval
+ Graph Traversal
↓
Documents + Relevant Knowledge Subgraph
↓
LLM

This becomes useful for multi-hop questions.

For example:

"Which experts worked on projects related to authentication incidents, and what lessons were learned?"

Answering that question may require traversing:

Authentication
→ Documents
→ Projects
→ Experts
→ Lessons

That is different from simply retrieving the five most semantically similar text chunks.

The PRD itself positions Knowledge Graph within the product's **"Connect"** level alongside Expert Discovery and Decision Intelligence, reinforcing this role as a connecting layer rather than just another search interface.

13. Success Criteria

The first version should be considered successful if it can:

1. represent major KMS knowledge objects as graph nodes;
2. normalize existing relationships into graph edges;
3. incorporate Rank 12 discovered relationships;
4. traverse at least one or two hops;
5. progressively expand the graph;
6. filter by relevant organizational context;
7. preserve relationship evidence/provenance;
8. enforce existing RBAC;
9. avoid requiring a separate graph database initially;
10. expose the graph through a reusable backend service/API.

Final Recommendation

Rank 13 should not be approached as **"build a graph UI."**

It should be approached as:

Build a reusable organizational knowledge graph layer first, then expose it through a useful visualization.

That distinction matters because the same graph layer can later support knowledge exploration, expert discovery, project intelligence, decision intelligence, GraphRAG and multi-hop organizational reasoning.

The resulting architecture would naturally progress from:

Rank 12: Discover the connections
↓
Rank 13: Organize and traverse the connections
↓
Future capabilities: Reason over the connections

This aligns well with the PRD's broader direction of moving the product from a Document Management System toward a **Knowledge Management System → Knowledge Intelligence Platform → Enterprise Knowledge Operating System.**